Campus Web House
Complete Project Diagram Set

20 requested diagram categories | 22 diagrams including DFD Levels 0, 1 and 2

Review preparation: 12 October 2026. Prepared 11 October 2026 from the implemented local project.

Stack: Django 5.2 backend, React/TypeScript interface, SQLite persistence, Vite/Tailwind asset build.

How to read: purple boxes are processes/components, teal boxes are data stores, grey boxes are external actors or supporting items. UML relationship meaning and cardinality are labeled directly where relevant.

Scope: student preferences use browser sessions; only admin login is implemented. No student signup, JWT, live AI service or external mentor delivery is shown. The Gantt chart is a proposed extension plan, not a record of dates actually worked.

Deliverables: a printable landscape PDF, 22 scalable SVG diagrams, editable generation source and a diagram manifest.

Diagram index

No.	Diagram	Priority	PDF page
01	Use Case Diagram	Essential	3
02	System Architecture Diagram	Essential	4
03a	DFD Level 0	Essential	5
03b	DFD Level 1	Essential	6
03c	DFD Level 2	Essential	7
04	Entity Relationship Diagram	Essential	8
05	Flowchart	Essential	9
06	Activity Diagram	Essential	10
07	Sequence Diagram	Essential	11
08	Class Diagram	Recommended	12
09	Component Diagram	Recommended	13
10	Deployment Diagram	Recommended	14
11	Database Schema Diagram	Essential	15
12	Wireframe Diagram	Essential	16
13	Sitemap	Essential	17
14	User Flow Diagram	Essential	18
15	State Diagram	Optional	19
16	API Flow Diagram	Recommended	20
17	Authentication Flow Diagram	Recommended	21
18	Network Architecture Diagram	Optional	22
19	Gantt Chart	Academic / planning	23
20	Context Diagram	Recommended	24

01 | Use Case Diagram

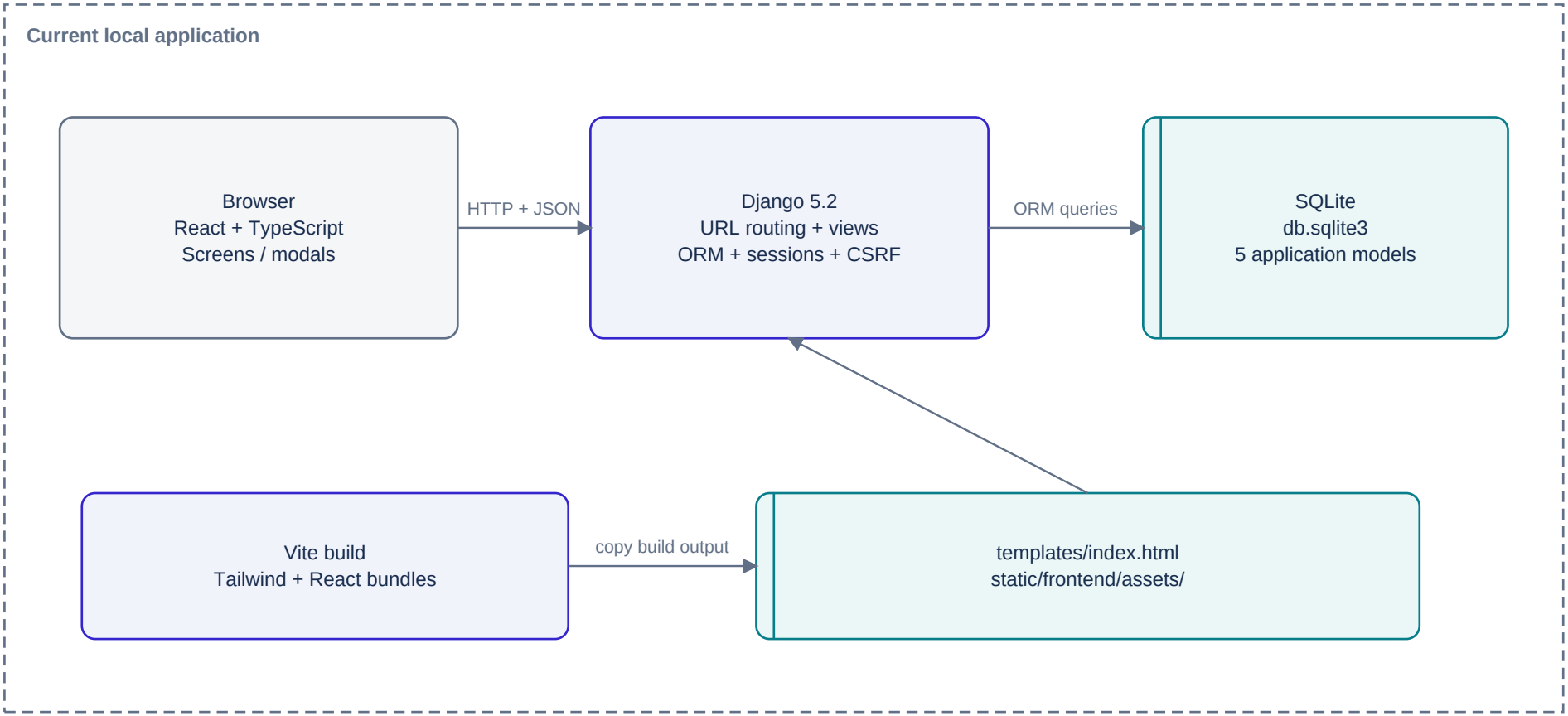
Priority: Essential



Student actions use browser sessions; no student signup exists. Admin management requires staff authentication and appropriate permissions. Career content and assessment remain demonstrations.

02 | System Architecture Diagram

Priority: Essential

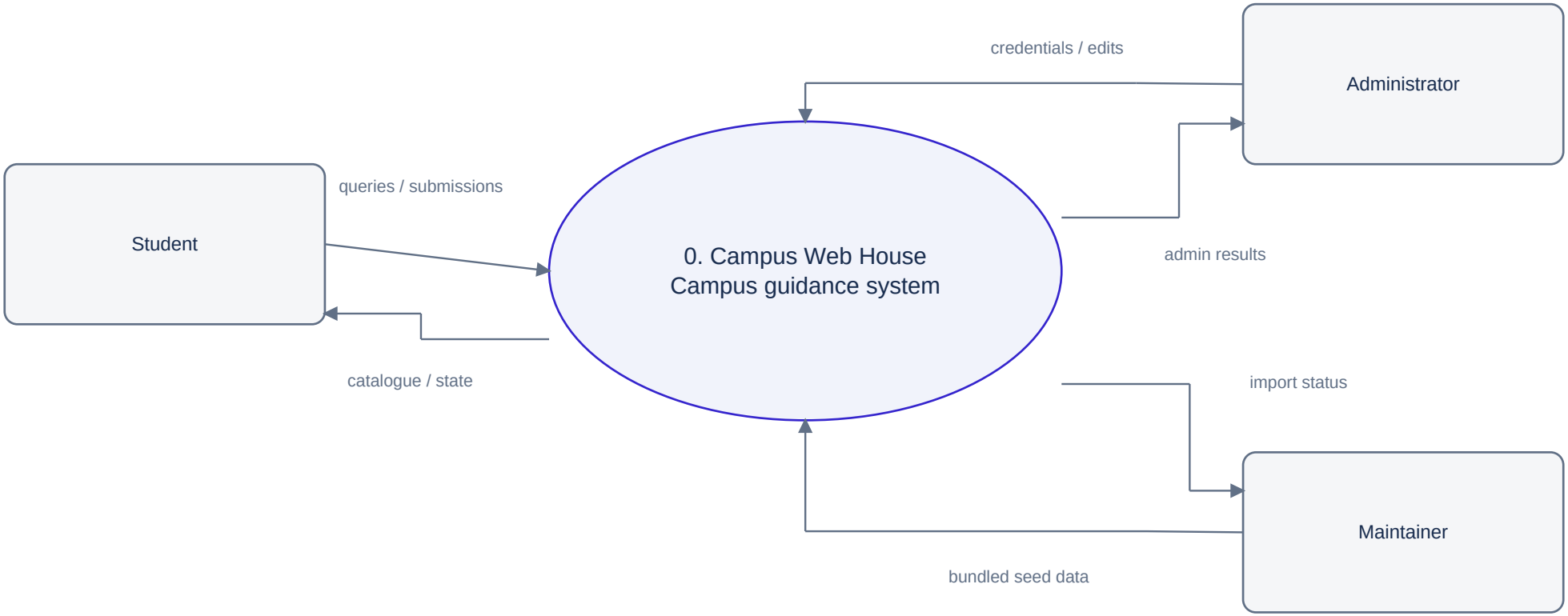


Node.js builds assets; Python serves the application.

The frontend and API share the Django origin in the built application. Vite is a development/build tool, not a required production API server.

03a | DFD Level 0

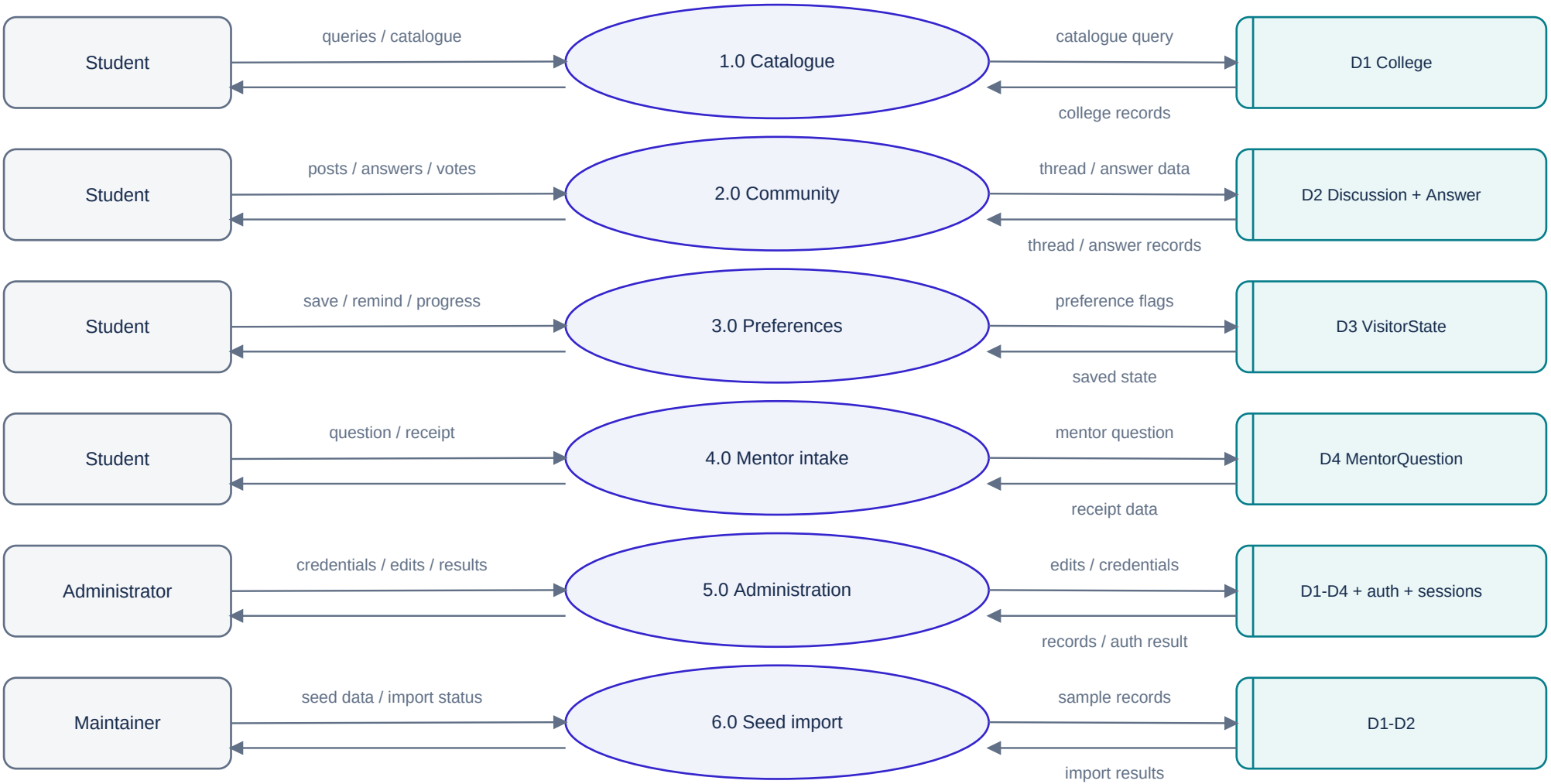
Priority: Essential



A single process shows system inputs and outputs. Student, administrator and maintainer are external actors. Internal tables and processes are intentionally hidden at this level.

03b | DFD Level 1

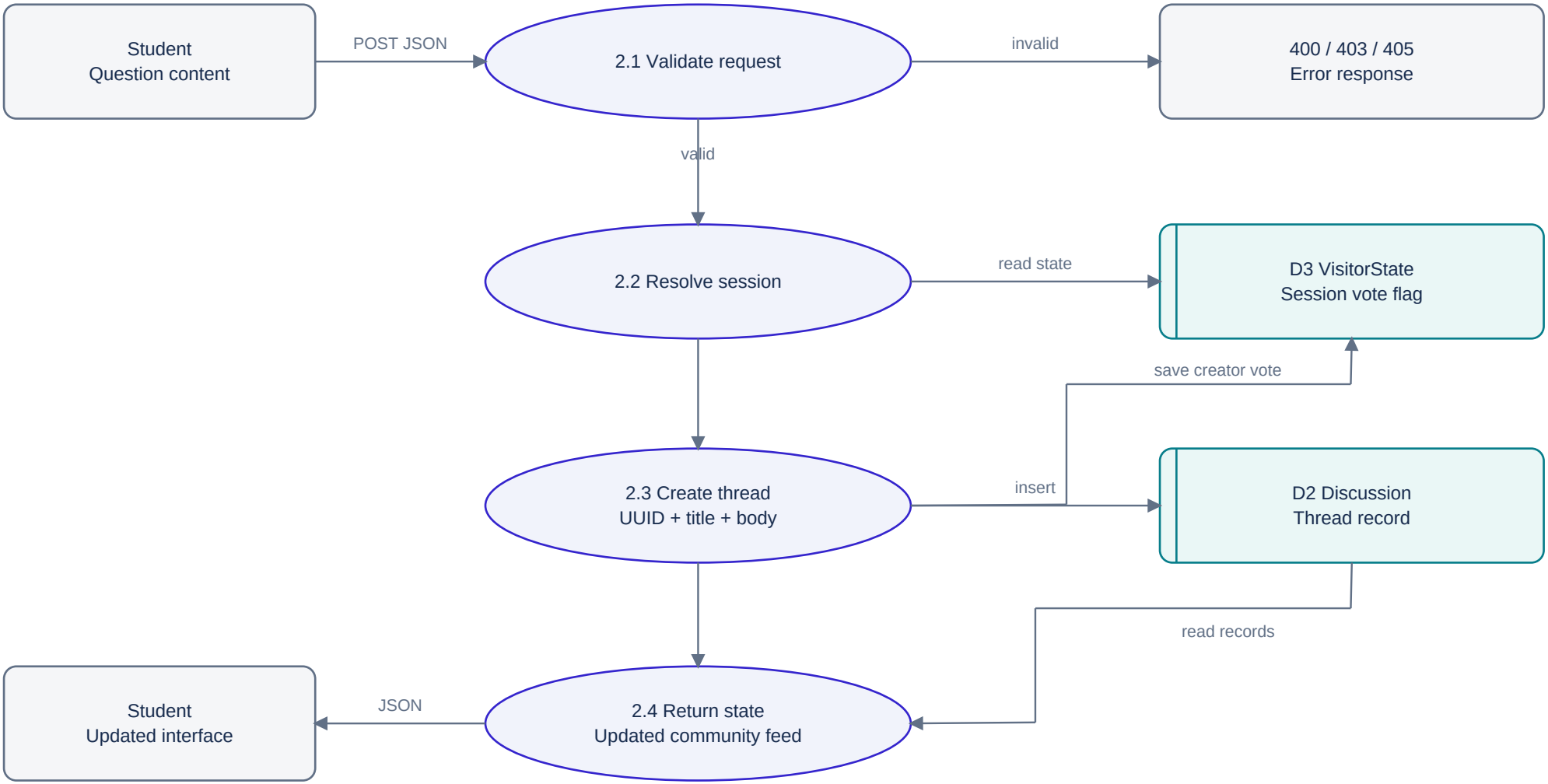
Priority: Essential



D1-D4 are logical stores implemented in one SQLite database. Paired arrows represent request/write data and returned/read data. Community and catalogue results merge with D3 preferences; administration uses Django auth/session tables. Seed import reads bundled JSON.

03c | DFD Level 2

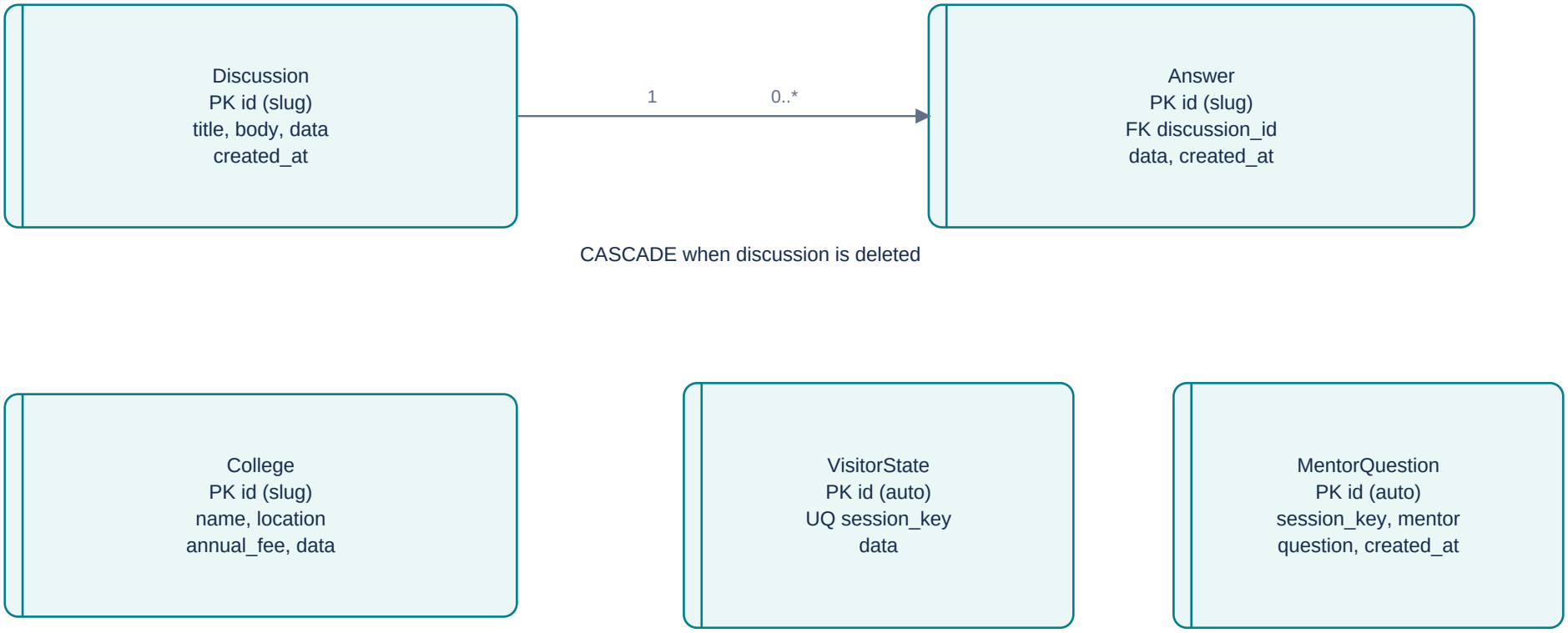
Priority: Essential



Decomposition of the create-discussion part of process 2.0. CSRF/method checks precede the view; title/body/tags are validated. The atomic action writes the thread and visitor vote state, then returns refreshed state.

04 | Entity Relationship Diagram

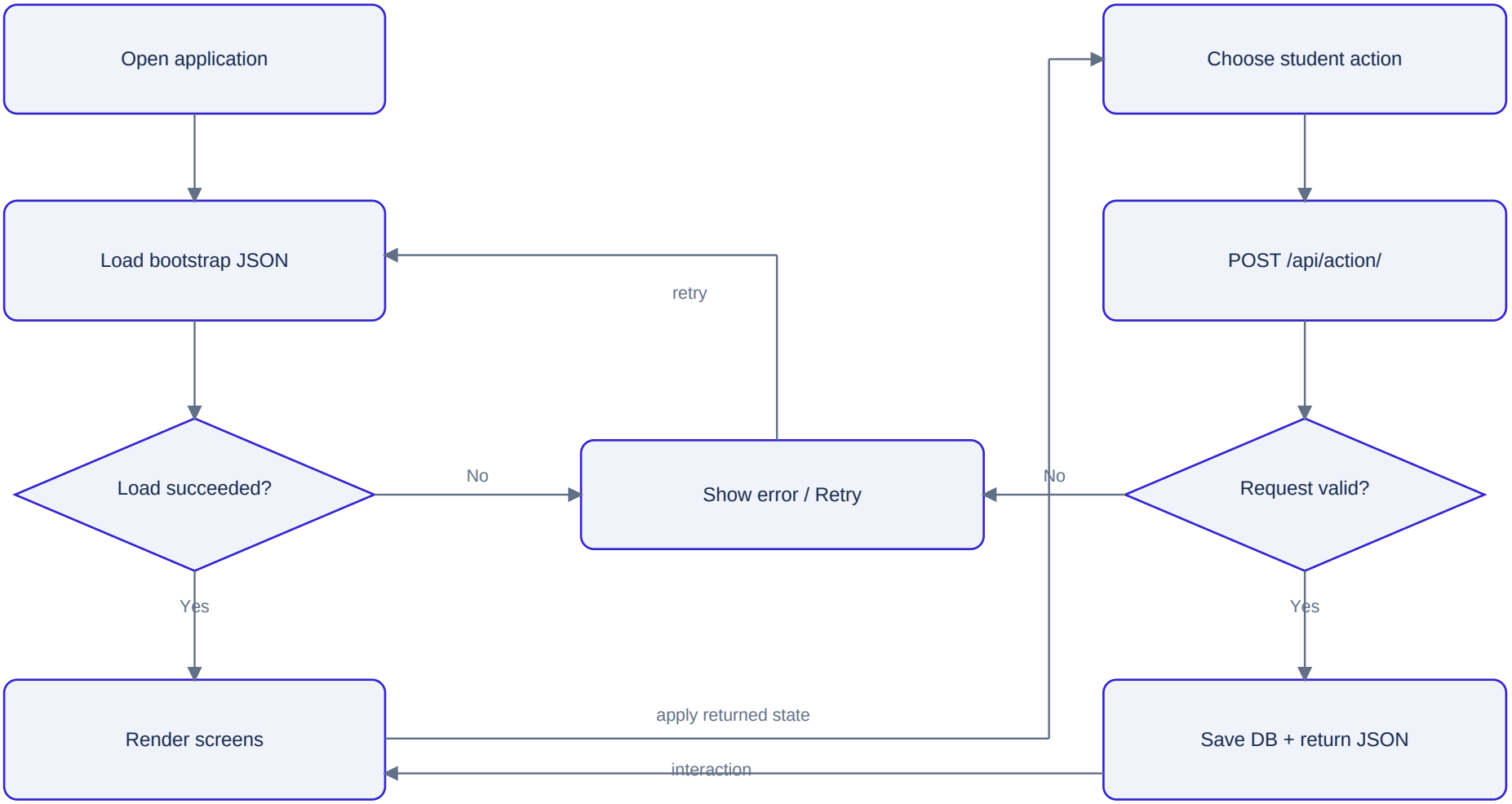
Priority: Essential



Only Discussion-to-Answer is an application-model foreign key. Saved college/thread identifiers inside VisitorState JSON are not enforced relational links. No College-to-Discussion or User-to-VisitorState foreign key exists.

05 | Flowchart

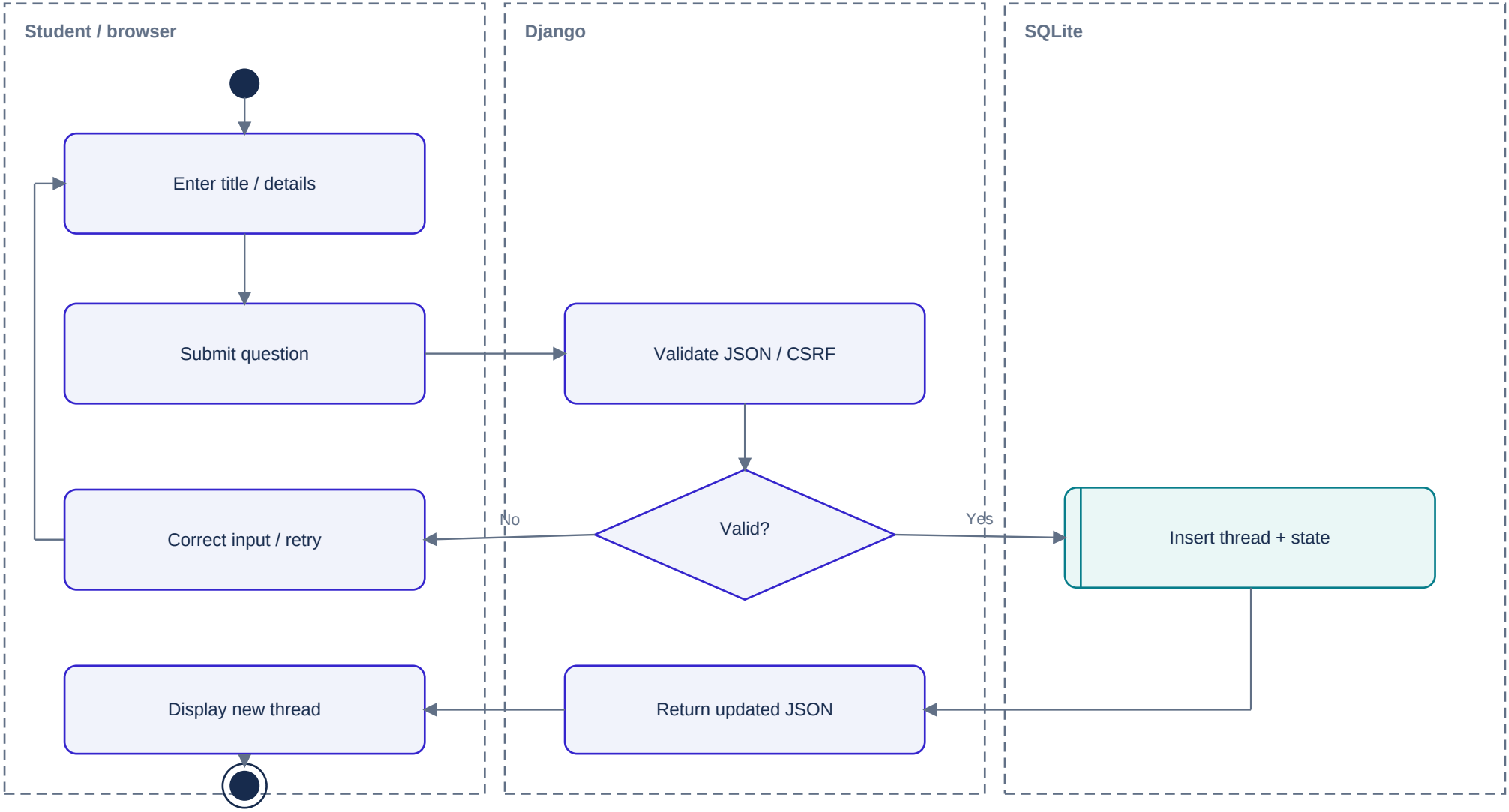
Priority: Essential



High-level page load and persisted-action flow. Failed POST requests display an error without claiming a successful save. Browser-only tools such as comparison do not require a write request.

06 | Activity Diagram

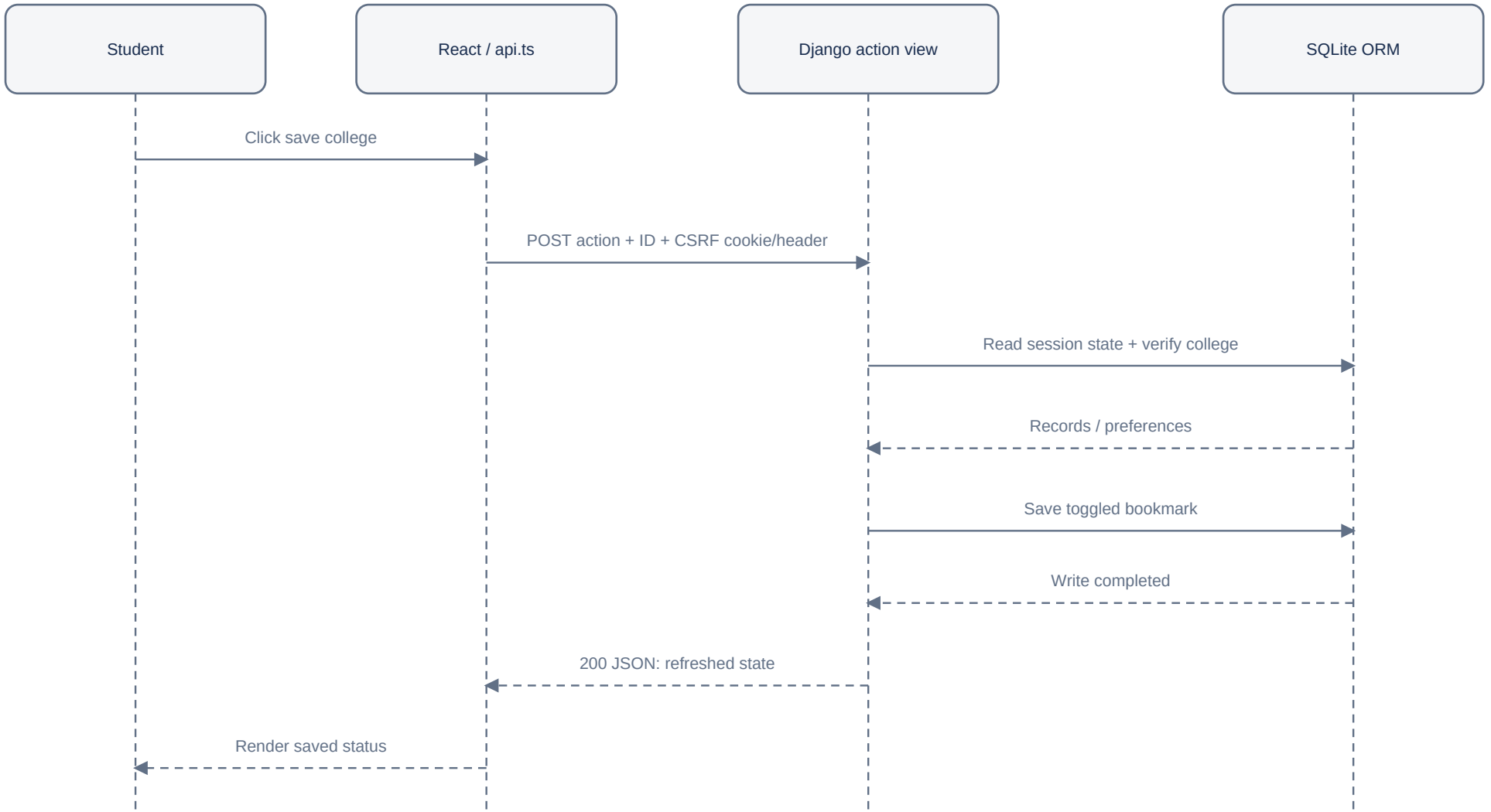
Priority: Essential



Swimlanes assign responsibilities. The happy path validates and saves a new community discussion. The invalid path returns an error and allows correction; no signup is required in the current prototype.

07 | Sequence Diagram

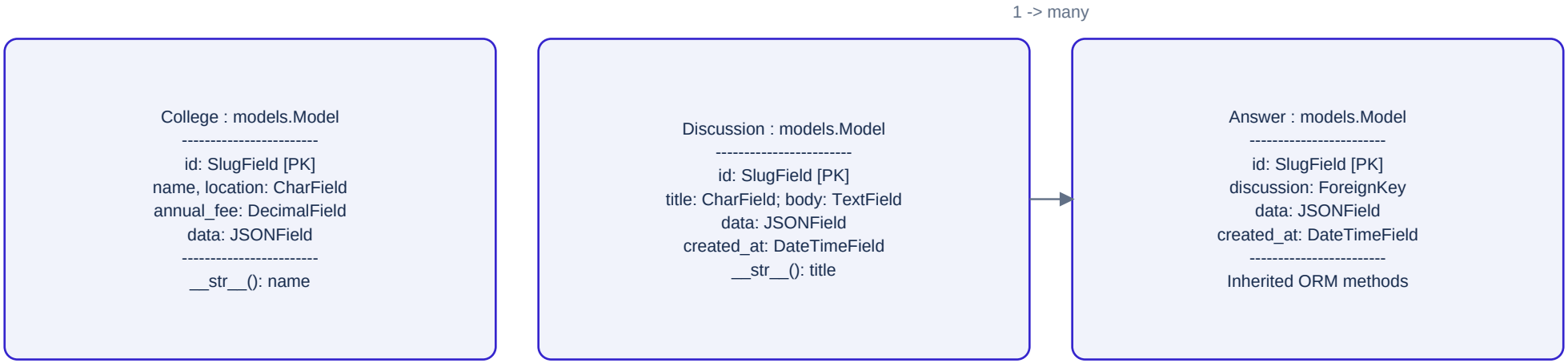
Priority: Essential



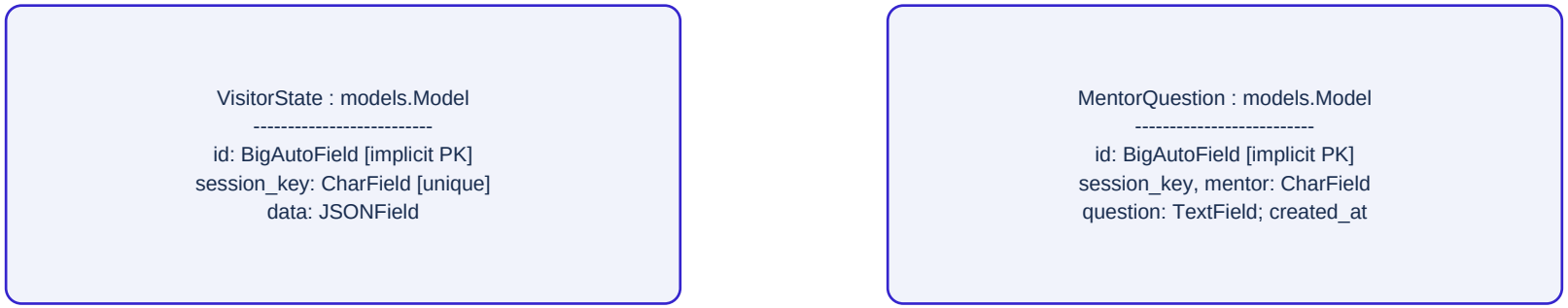
The save-college sequence represents successful interaction order. Responses use dashed arrows. Refresh persistence follows from another bootstrap GET reading the same browser session. Error sequences are summarized in the API/authentication diagrams.

08 | Class Diagram

Priority: Recommended



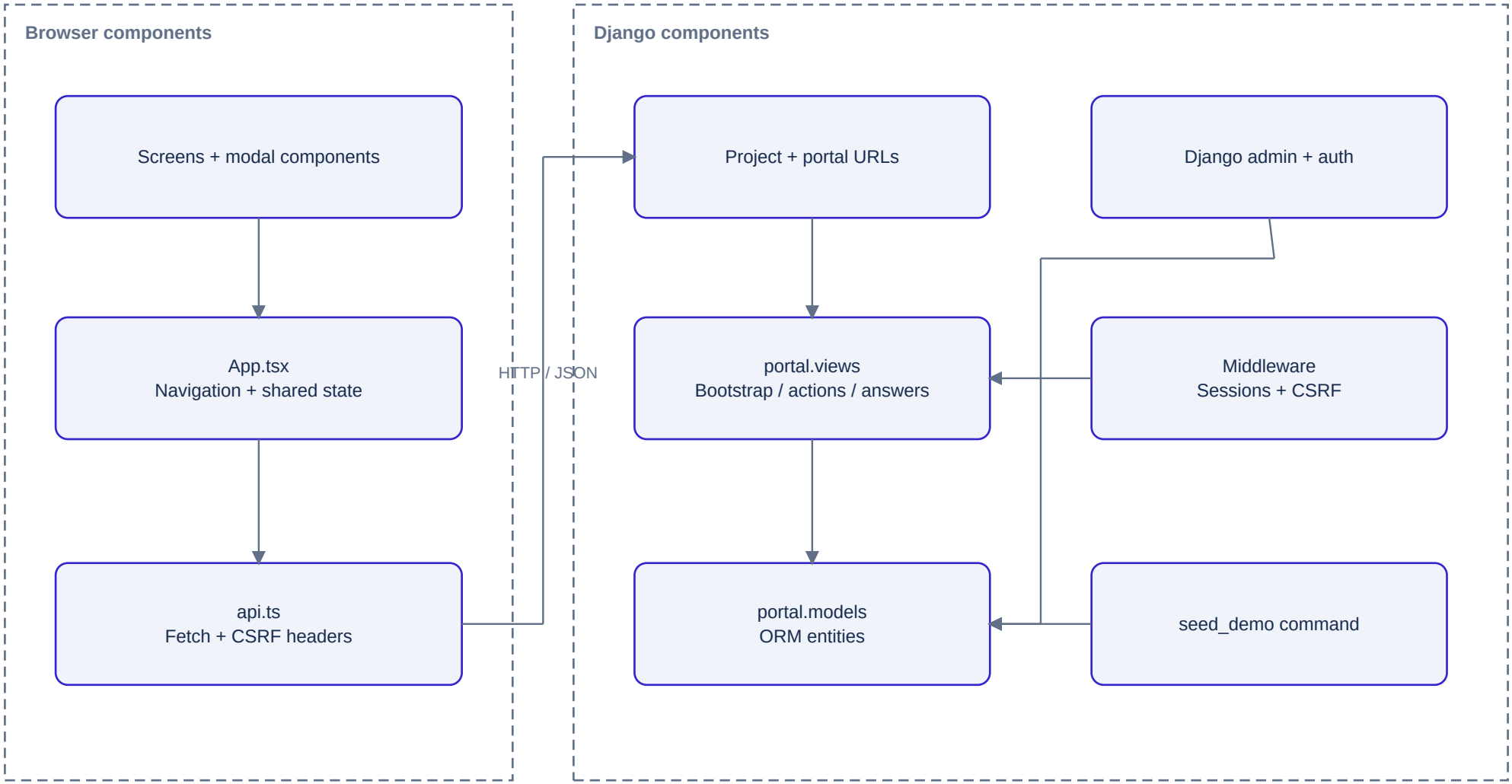
All classes inherit save(), delete() and the objects manager.



The diagram mirrors the actual Django model classes. Only College and Discussion define __str__. save/delete/query APIs are inherited, not custom methods. There are no fabricated student login or recommendation classes.

09 | Component Diagram

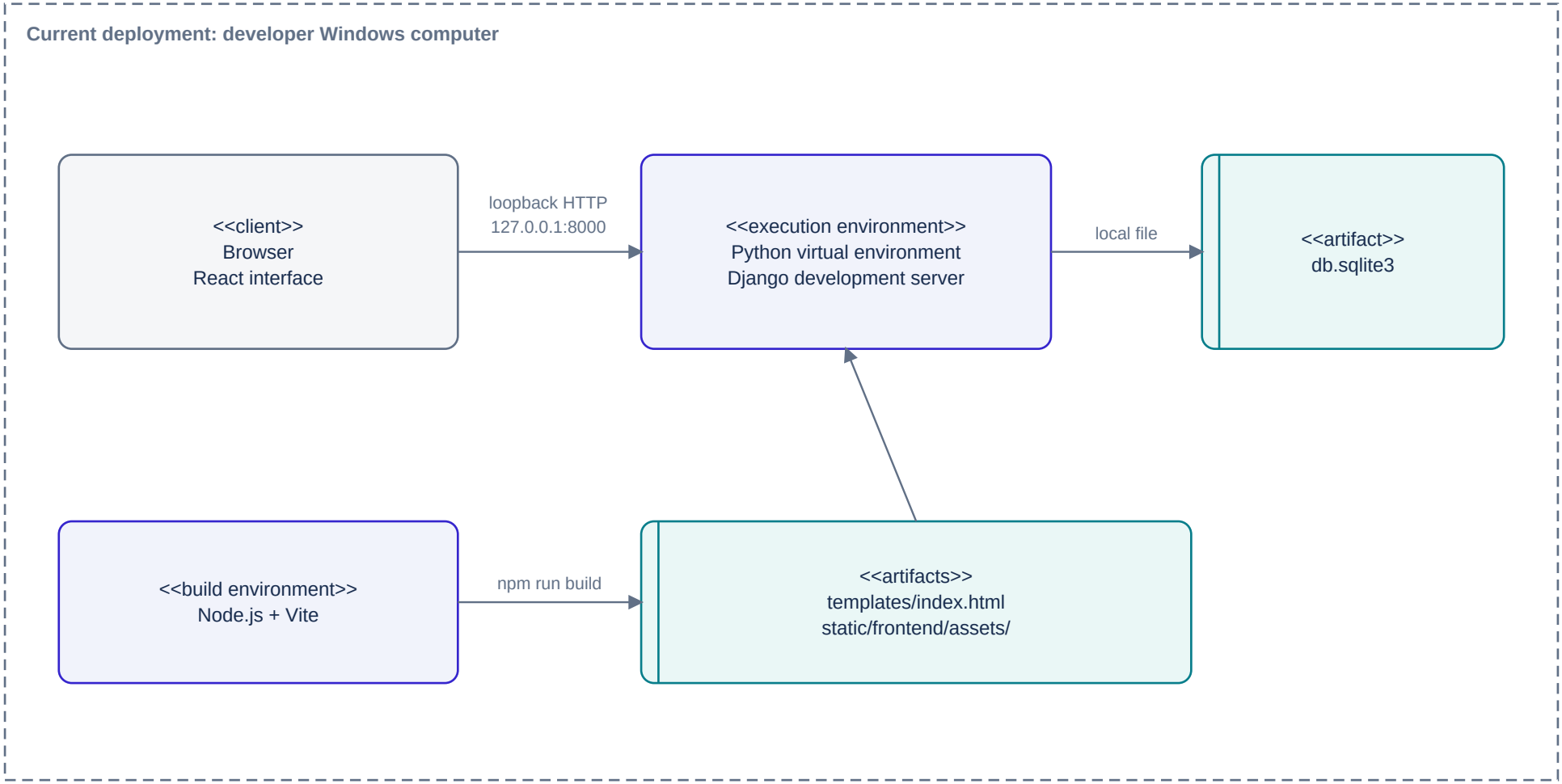
Priority: Recommended



Component responsibilities are dependencies, not separate deployed services. React has no direct database access. The seed command and admin also use Django models. Framework middleware participates in the request lifecycle.

10 | Deployment Diagram

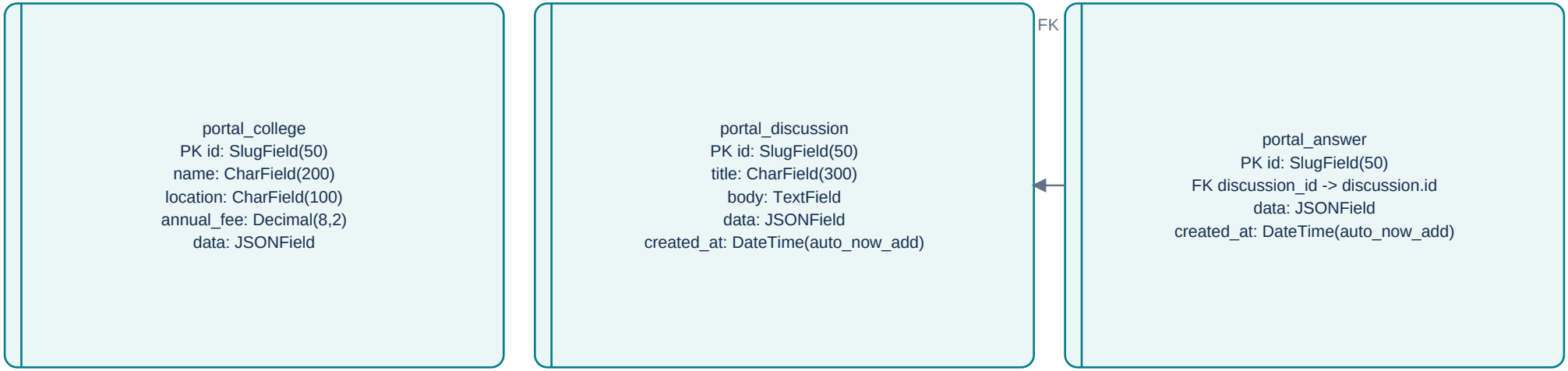
Priority: Recommended



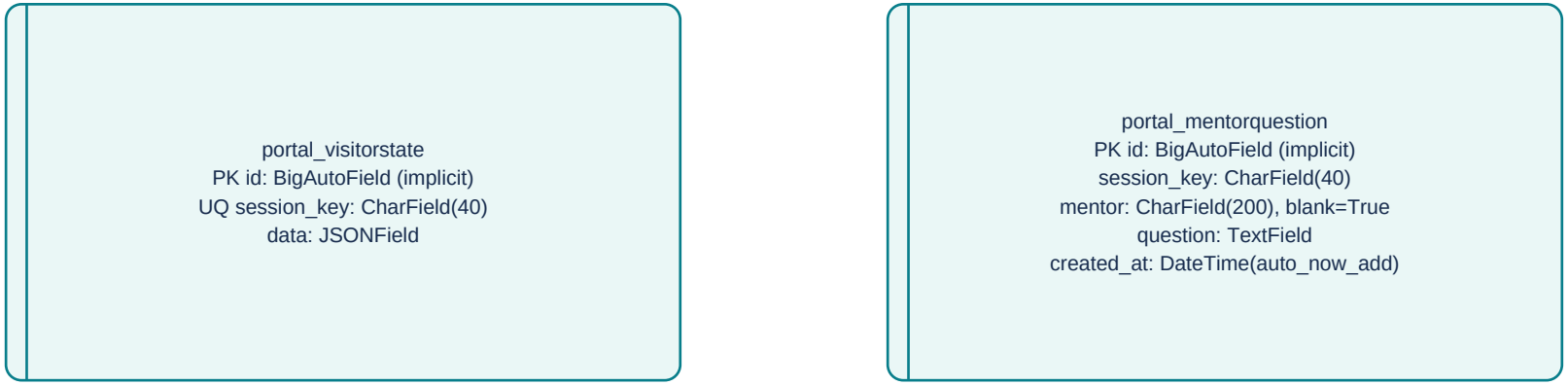
Actual deployment is local, not cloud hosting. .venv is outside Campus. A production deployment would need an appropriate WSGI/ASGI server, HTTPS and separate static-file serving; these are future configuration, not currently implemented nodes.

11 | Database Schema Diagram

Priority: Essential



Fields shown as Django schema types; all 5 application tables.



All application fields are included. Session key fields are not database foreign keys. Django additionally manages auth, session, permission, content-type, migration and admin-log tables; those framework tables are listed in the accompanying README.

12 | Wireframe Diagram

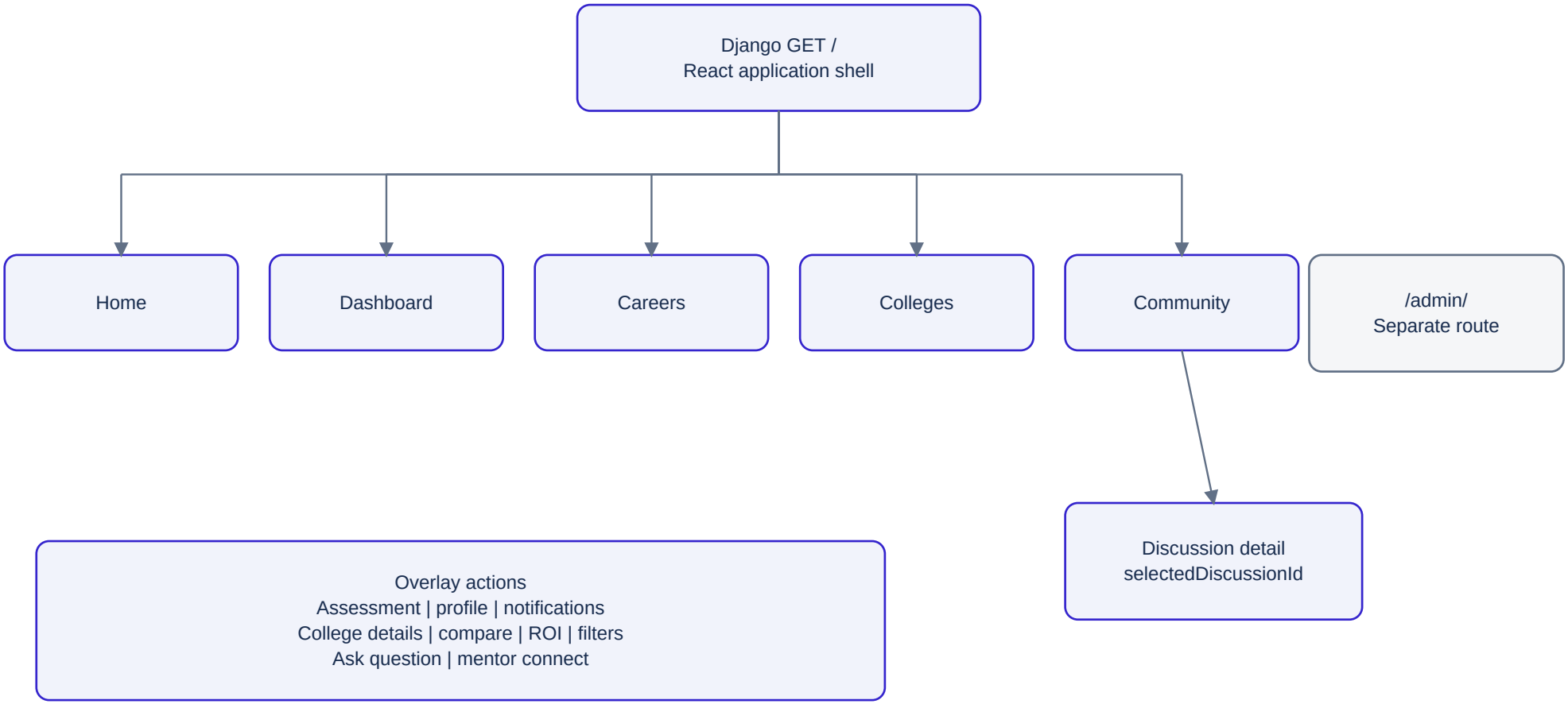
Priority: Essential



Low-fidelity structural wireframes of three existing screens. This is not a pixel-exact screenshot. Shared header and mobile bottom navigation remain outside screen content; dialogs/drawers appear as overlays.

13 | Sitemap

Priority: Essential

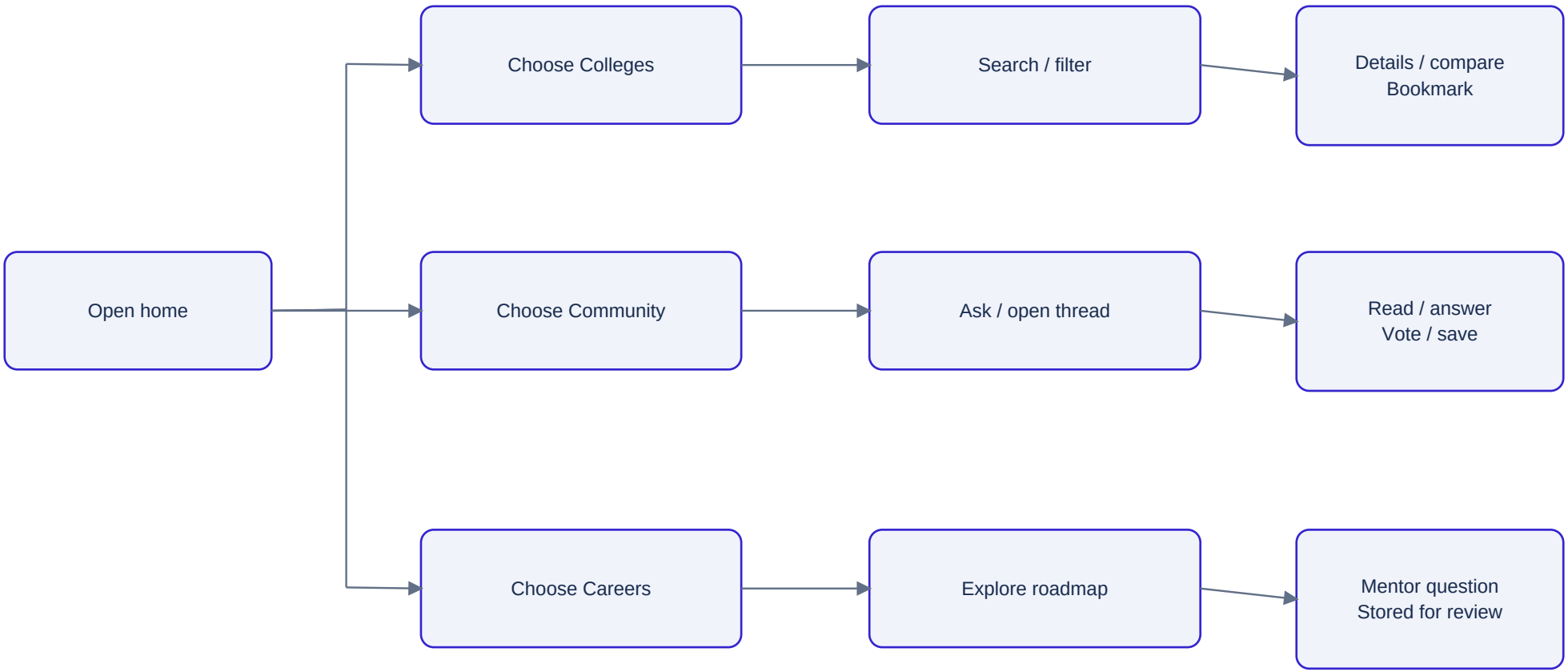


Screen names are React states, not independent Django page URLs.

Logical screen hierarchy for the SPA. Dashboard, careers, colleges and community are React views under /. Admin is a distinct Django route. The overlays are application dialogs, not additional website pages.

14 | User Flow Diagram

Priority: Essential

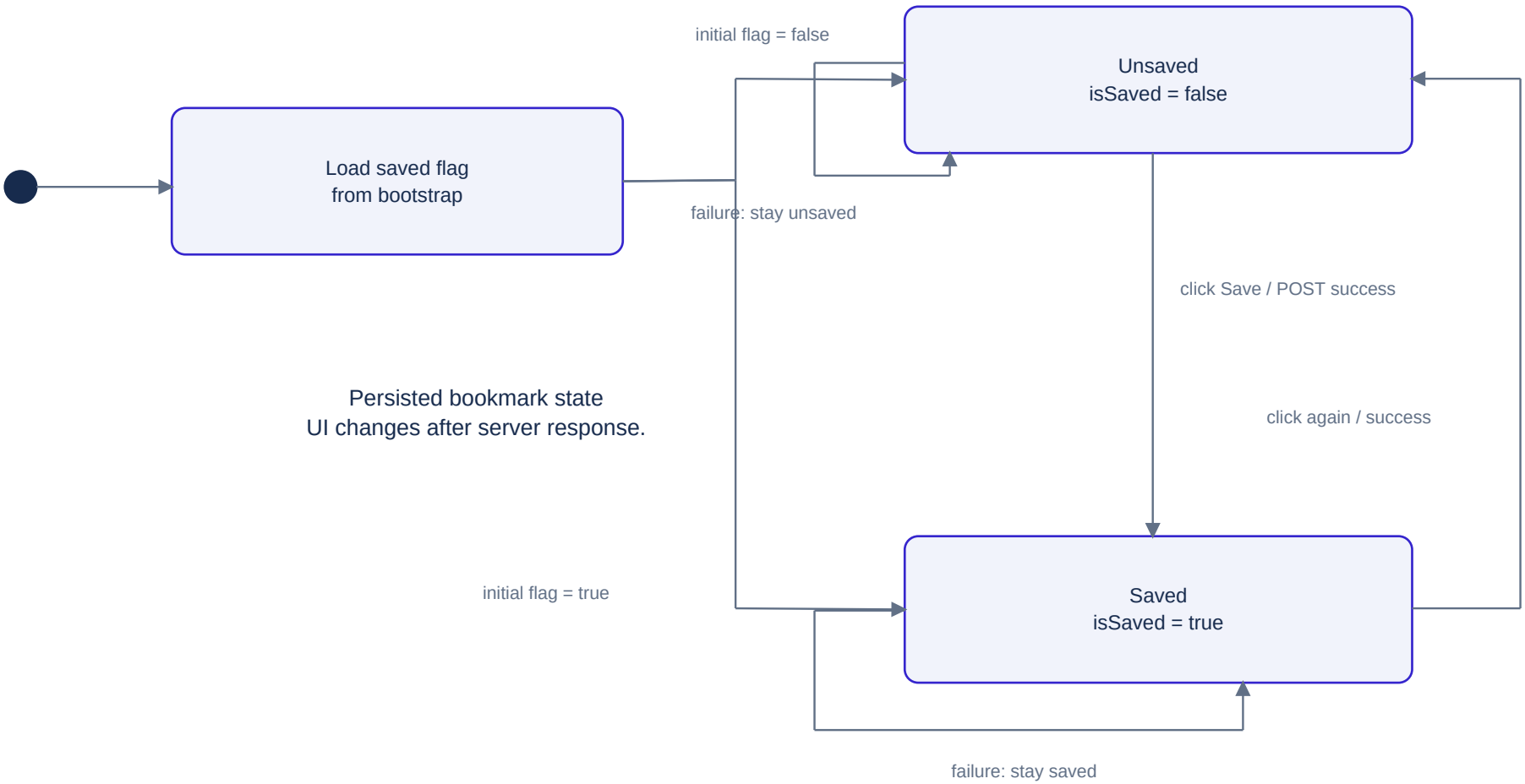


No student login step is implemented in these journeys.

Shows three real navigation journeys. Only actions wired to the API persist. Comparison selection and roadmap task state remain temporary UI state; mentor questions are stored without external delivery.

15 | State Diagram

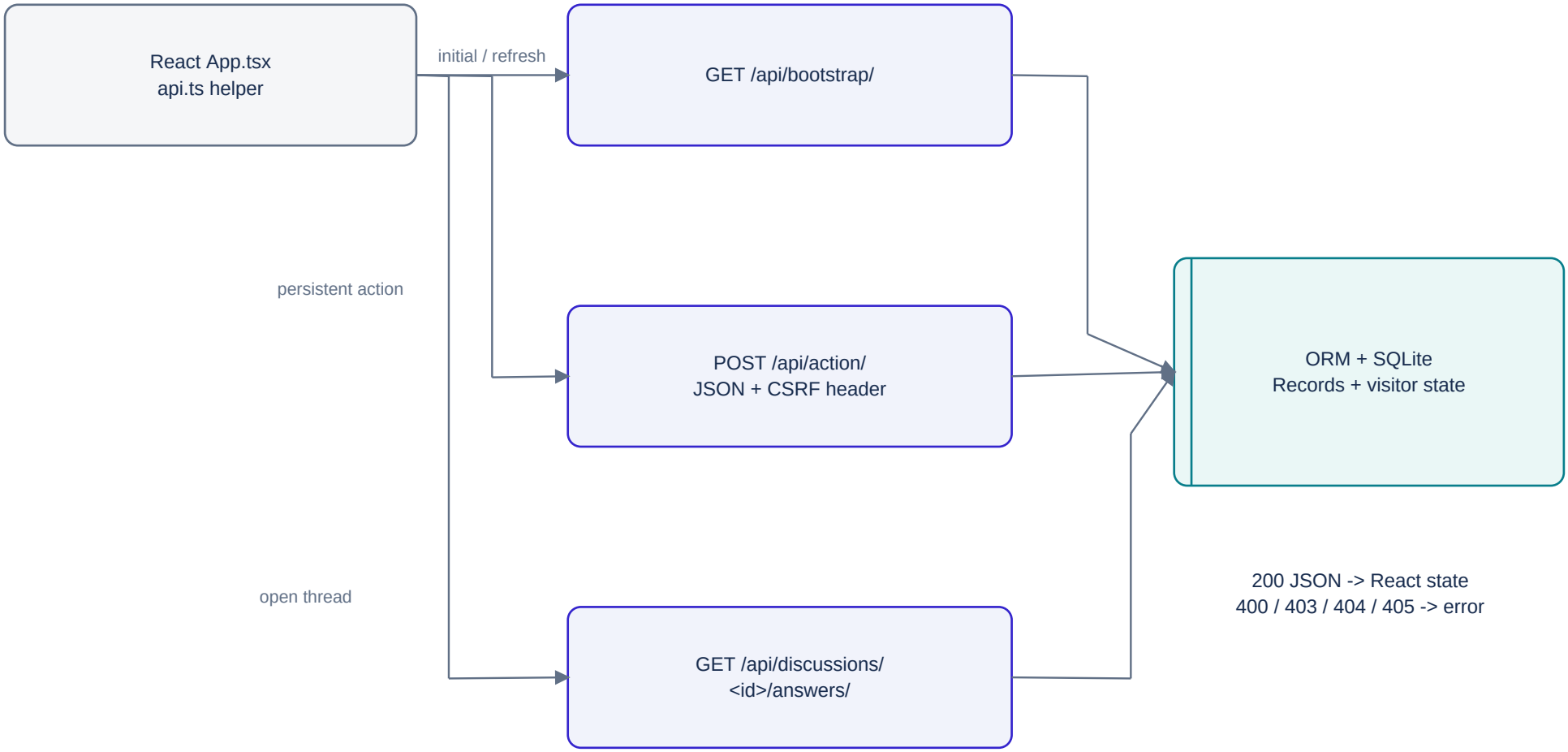
Priority: Optional



State machine for one college bookmark, not a whole student profile. Initial state comes from session-aware bootstrap. A successful POST toggles the stored flag. An unsuccessful request leaves the last confirmed state unchanged.

16 | API Flow Diagram

Priority: Recommended

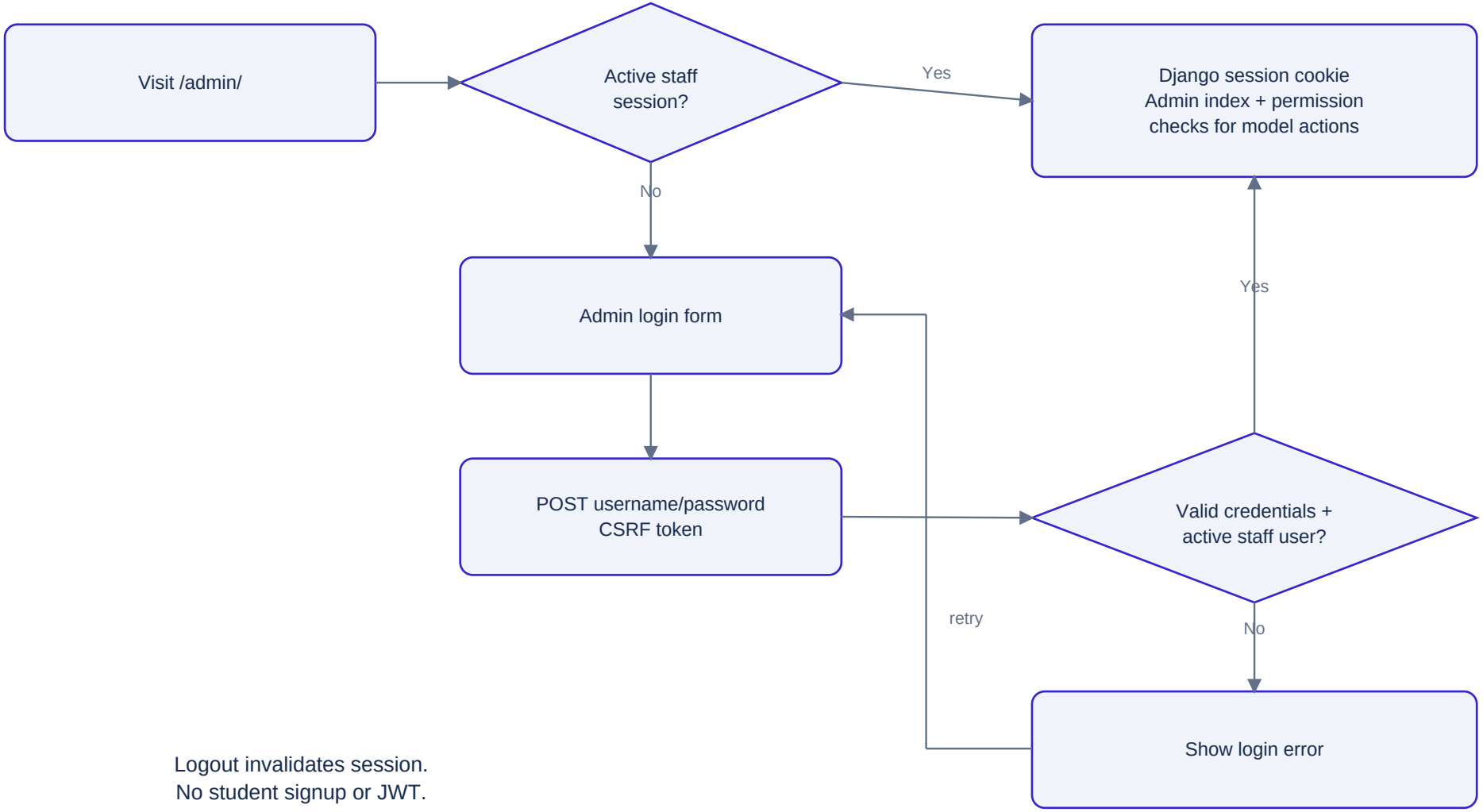


Frontend and API use the same origin in the built Django application.

api.ts handles bootstrap and action calls; DiscussionDetailScreen fetches the thread-answer endpoint directly. Normal responses contain refreshed state or answer data. Error responses must not be interpreted as successful database saves.

17 | Authentication Flow Diagram

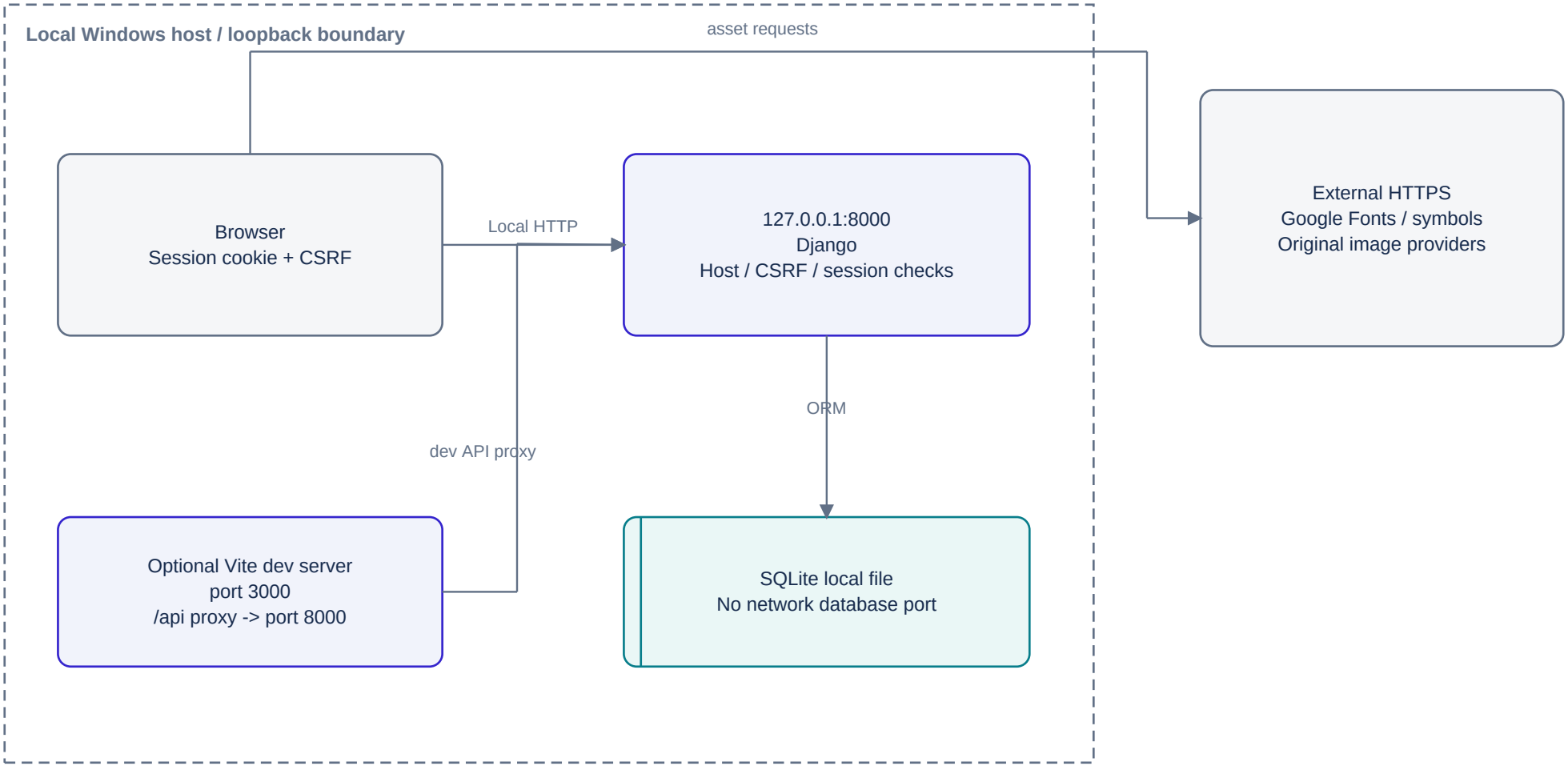
Priority: Recommended



This is Django admin session authentication, not JWT or student authentication. Django checks active/staff status and model permissions; a superuser has broad permissions. Student API actions currently use visitor sessions, not registered student ownership.

18 | Network Architecture Diagram

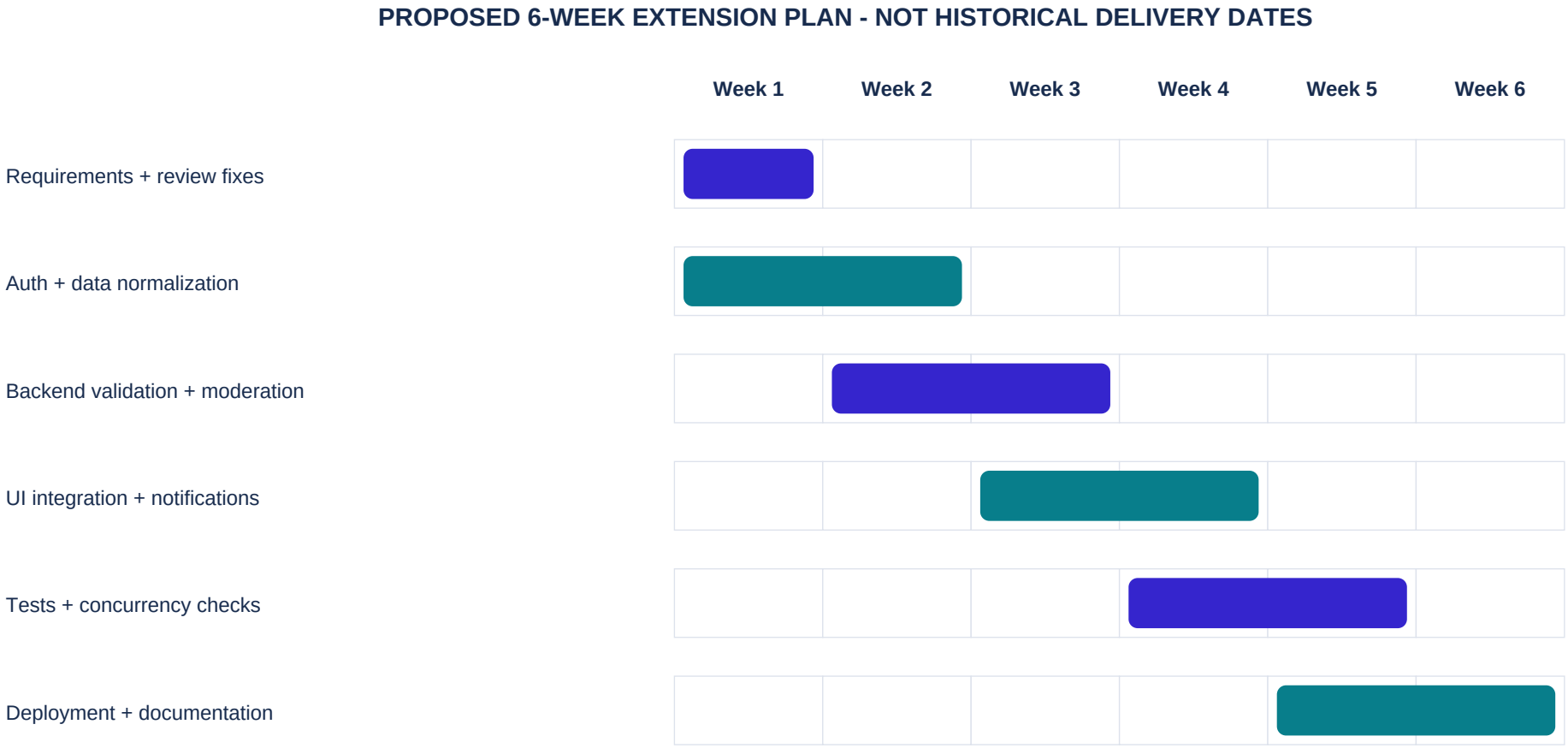
Priority: Optional



The implemented environment uses local HTTP on loopback. External font/image requests need internet. This is not a deployed internet service; HTTPS termination, firewall policy and production hosting are future infrastructure. No JWT or cloud database is present.

19 | Gantt Chart

Priority: Academic / planning

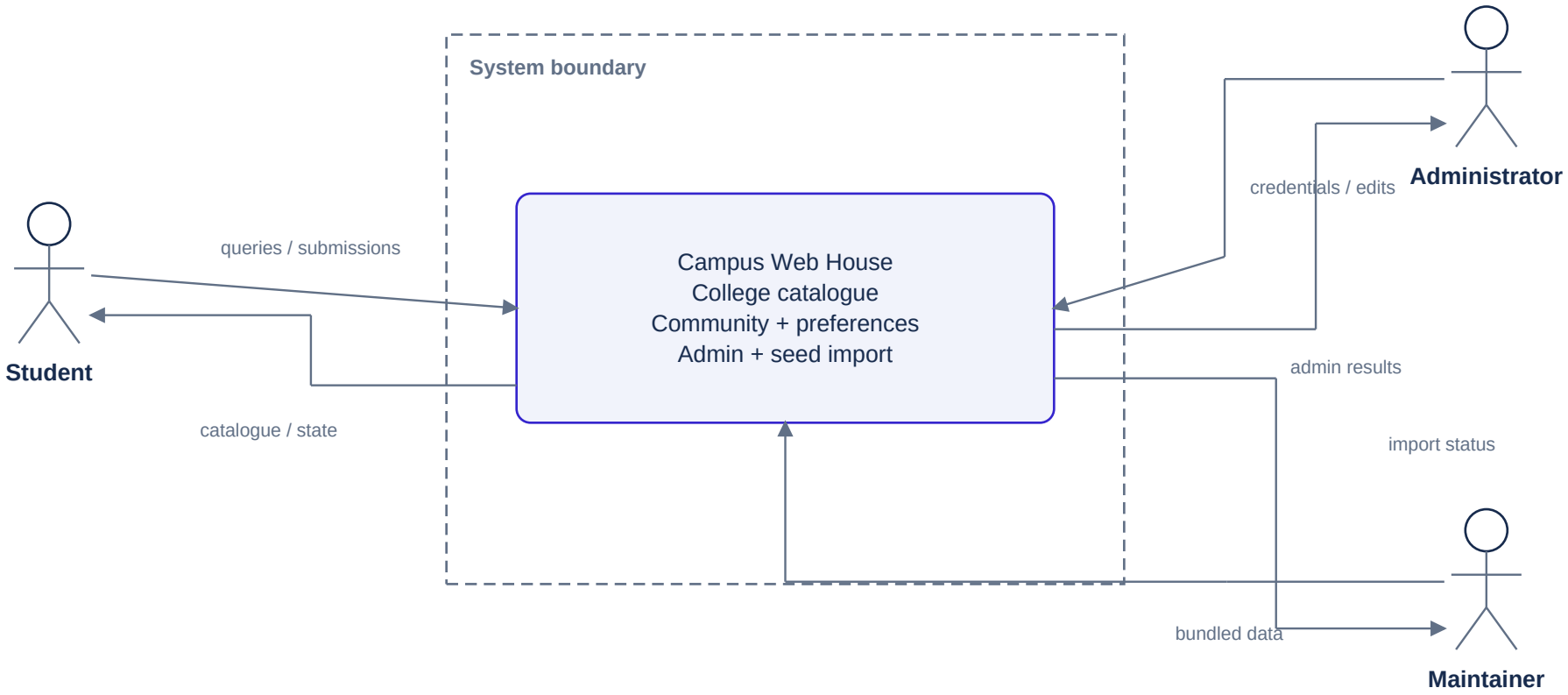


Sequence is a proposal; duration and resources must be agreed with the mentor.

No development schedule was provided. This chart is an explicitly proposed six-week improvement plan, with relative weeks rather than invented completion dates. Scope and timing are subject to mentor review.

20 | Context Diagram

Priority: Recommended



Context and DFD Level 0 intentionally cover the same system boundary. This view emphasizes external actors, while the DFD labels the single process as 0. No mentor-delivery platform, payment service or AI API is connected.